

Problem 6 – Smart Expense Tracker 💰

Difficulty: ★★ Beginner

Overview

A university student wants to better manage their daily spending. Instead of writing expenses on paper, they would like a simple application that records each expense, keeps track of the total amount spent, and displays useful spending summaries.

Your task is to build a Smart Expense Tracker using Streamlit.

Project Requirements

1. Expense Categories

Create a dictionary that stores the available expense categories.

Category

Food

Transportation

Shopping

Entertainment

Bills

2. Store Expenses

Store all expenses in a list.

Each expense must contain:

- Expense Name
- Category
- Amount (EGP)
- Expense Date

Each expense can be stored as a dictionary.

3. Build the User Interface

Use Streamlit to create the application.

Required Text Functions

Use all of the following:

- `st.title()`
 - `st.header()`
 - `st.subheader()`
 - `st.write()`
 - `st.divider()`
-

4. Expense Entry Form

Create a form that allows the user to enter a new expense.

Use the following widgets:

Widget	Purpose
<code>st.text_input()</code>	Expense Name
<code>st.selectbox()</code>	Category
<code>st.number_input()</code>	Amount
<code>st.date_input()</code>	Expense Date
<code>st.button()</code>	Add Expense

5. Validate User Input

Before saving an expense, make sure that:

- Expense name is not empty.
- Amount is greater than zero.

If the input is invalid, display an appropriate error message.

6. Save Expenses

When the Add Expense button is clicked:

- Validate the data.
 - Store the expense in the expense list.
 - Display a success message.
-

7. Display the Expense List

Create a section that displays every recorded expense.

Example

Lunch

Category : Food

Amount : 120 EGP

Date : 10/08/2025

Taxi

Category : Transportation

Amount : 80 EGP

Date : 10/08/2025

8. Display the Expense Summary

Calculate and display:

- Total number of expenses
- Total amount spent

Example

Summary

Total Expenses : 2

Total Spending : 200 EGP

Required Status Elements

Use all of the following:

- `st.info()`
- `st.success()`
- `st.warning()`
- `st.error()`

Suggested usage:

- Info: Budget or spending tips.
 - Success: Expense added successfully.
 - Warning: Expense exceeds 1000 EGP.
 - Error: Invalid expense name or amount.
-

Required Layout

Sidebar

Display:

- Today's Date
 - Expense Categories
 - Budget Tips
-

Two Columns

Left Column

Display the expense entry form.

Right Column

Display:

- Expense Summary
 - Total Spending
 - Number of Expenses
-

Tabs

Create three tabs.

Tab 1

Add Expense

Contains the expense entry form.

Tab 2

Expense List

Displays all recorded expenses.

Tab 3

Summary

Displays:

- **Total number of expenses**
 - **Total spending**
-

Bonus Mission ★

Display the total amount spent for each category.

Example

Food : 350 EGP

Transportation : 120 EGP

Shopping : 450 EGP

Entertainment : 90 EGP

Bills : 700 EGP

Challenge Mission ★★ – Daily Spending Report

Create a Daily Spending Report that analyzes all recorded expenses and displays the following:

- Highest Expense: The expense with the largest amount, including its name and amount.
- Lowest Expense: The expense with the smallest amount, including its name and amount.
- Average Expense: The average amount spent per expense.
- Category with the Highest Spending: The category whose total spending is the greatest.

Example

Daily Spending Report as a table

Highest Expense:

Laptop Bag - 850 EGP

Lowest Expense:

Water - 15 EGP

Average Expense:

246.25 EGP

Category with the Highest Spending:

Shopping - 1,250 EGP

Notes

- The report should be calculated using all recorded expenses.
- If no expenses have been added yet, display an appropriate message instead of generating the report.

This wording removes ambiguity by making it clear that:

- Highest/Lowest Expense refer to a single expense record, not a category.
- Category with the Highest Spending is based on the sum of all expenses in each category.
- Average Expense is calculated as:

Average Expense = Total Spending ÷ Number of Expenses

Useful Hints

- Store expenses as a list of dictionaries.
 - Create reusable functions such as:
 - `add_expense()`
 - `display_expenses()`
 - `calculate_total()`
 - `category_summary()`
 - Use `datetime.today()` to display today's date.
 - Use `st.session_state` to keep expenses while the application is running.
-

Expected GUI

Smart Expense Tracker

Sidebar

Today's Date

Expense Categories

Budget Tips

Tabs

[Add Expense] [Expense List] [Summary]

Expense Name

[Lunch ▼]

Category

[Food ▼]

Amount

[120]

Date

[10/08/2025]

[Add Expense]

Summary

Total Expenses : 1

Total Spending : 120 EGP

☒ Expense added successfully!

Python Topics Covered

- Variables
 - Strings
 - Integers
 - Floats
 - Lists
 - Dictionaries
 - Arithmetic Operators
 - Comparison Operators
 - Conditional Statements (if, elif, else)
 - User-defined Functions
 - datetime
 - st.session_state
-

Streamlit Topics Covered

Display

- `st.title()`
- `st.header()`
- `st.subheader()`
- `st.write()`
- `st.divider()`

Input Widgets

- `st.text_input()`
- `st.selectbox()`
- `st.number_input()`
- `st.date_input()`
- `st.button()`

Status Elements

- `st.info()`
- `st.success()`
- `st.warning()`
- `st.error()`

Layout

- `st.sidebar`
- `st.columns()`
- `st.tabs()`

State Management

- `st.session_state`
-

Learning Objectives

After completing this project, you should be able to:

- Store structured data using a list of dictionaries.
- Validate user input before storing data.
- Work with dates in Python.
- Calculate totals and spending summaries.
- Organize a program into reusable functions.
- Build a well-organized multi-tab Streamlit application using sidebars, columns, and session state.